

Conversation with Gemini 9/2/2026

gleam wisp vs bun typescript hono for web services point by point comparison including dev experience, performance, tooling, etc.

Choosing between **Gleam (Wisp)** and **Bun (TypeScript + Hono)** for web services depends on your architectural needs: if you prioritize **fault tolerance and massively scalable concurrency**, select **Gleam + Wisp**; if your goal is **lightning-fast raw execution speed, absolute minimal cold starts, and a massive library ecosystem**, choose **Bun + Hono**. [1, 2, 3, 4, 5]

Quick Comparison

Feature	 Gleam + Wisp	 Bun + TypeScript + Hono
Language Paradigm	Statically typed, functional	Statically typed (via TS), object-oriented/imperative
Runtime Engine	Erlang BEAM (or JS via Node/Bun compilation)	JavaScriptCore (WebKit engine)
Concurrency Model	Actor Model (isolated light-weight processes)	Event Loop (async/await tasks)
Type System	Sound type system (no nulls, no exceptions)	Structural type system (types stripped at runtime)
Routing Approach	Explicit Pattern Matching	High-speed Trie-based RegExpRouter
Ecosystem Size	Small, growing niche (can interop with Erlang/Elixir)	Massive (native access to millions of npm packages)

Architectural Philosophy

- **Gleam + Wisp:** Built for **resilience**. It leverages the [Erlang BEAM VM](#) to manage web traffic using isolated processes. A crash in one request handler cannot bring down other requests or the application server. [1, 2, 3]

- **Bun + Hono:** Built for **speed and ubiquity**. It adheres strictly to modern web standards (Fetch API). It allows you to run identical API code seamlessly across the edge, serverless platforms, or traditional VPS infrastructure. [[1](#), [2](#), [3](#), [4](#), [5](#), [6](#)]
-

Developer Experience & Type Safety

- **Gleam + Wisp DX:**
 - **Strict Type Safety:** Features a highly rigid compiler with no implicit type casting or null values.
 - **No Magic:** Enforces explicitly passed Context objects (for DB pools, config) into request handlers instead of relying on global dependency injection.
 - **Pattern Matching:** Maps HTTP routes cleanly using structural code matching rather than strings or decorators. [[1](#), [2](#), [3](#), [4](#)]
 - **Bun + Hono DX:**
 - **Flexible Types:** Provides highly comprehensive autocomplete using a structural type system.
 - **Hono RPC:** Shares end-to-end type safety directly with frontend frameworks without running code-generation steps.
 - **Familiar Syntax:** Leverages mainstream Express-like middleware syntax for instant familiarity. [[1](#), [2](#), [3](#), [4](#)]
-

Performance & Concurrency

- **Gleam + Wisp Performance:**
 - **High Concurrency:** Scales exceptionally well under heavy parallel request spikes.
 - **Predictable Latency:** Utilizes a pre-emptive scheduler to ensure long-running database tasks never freeze the server thread.
 - **Fault Recovery:** Includes built-in supervisor trees to automatically heal failing background processes. [[1](#)]
- **Bun + Hono Performance:**

- **Raw Throughput:** Executes CPU-bound tasks and basic routing loops faster than the BEAM.
 - **Optimized Router:** Employs a zero-dependency [RegExpRouter](#) to achieve microsecond route lookups.
 - **Low Overhead:** Boasts sub-50ms cold starts, making it perfectly optimized for serverless functions. [[1](#), [2](#), [3](#)]
-

Tooling & Ecosystem

- **Gleam Tooling:**
 - **Unified CLI:** Includes built-in formatters, build pipelines, and test runners directly inside the single gleam binary.
 - **Ecosystem Limits:** Relies on a smaller pool of native web packages, occasionally requiring you to write custom boilerplate to bridge Elixir or Erlang tools. [[1](#), [2](#)]
 - **Bun Tooling:**
 - **All-in-One Toolkit:** Combines a package manager, bundler, test framework, and TS runner natively.
 - **Ultra-fast Installs:** Resolves and installs packages up to 30x faster than traditional npm tools.
 - **Limitless Libraries:** Grants out-of-the-box support for the extensive npm ecosystem, including robust ORMs like Drizzle. [[1](#), [2](#), [3](#), [4](#), [5](#)]
-

To help narrow down your stack choice, tell me about your project:

- What **database or third-party APIs** will this service connect to?
- Where do you plan to **deploy the application** (e.g., AWS, serverless edge, or a standard VPS)?
- How large is the **development team** and what is their language background?
- 

GitHub Pages documentation

Feasibility of bun/hono for web/routing layer and gleam javascript target for service layer

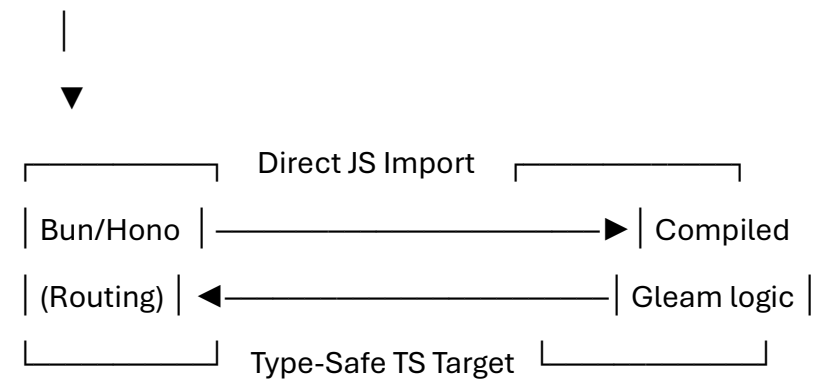
Using **Bun/Hono** for your web/routing layer combined with a **Gleam (JavaScript target)** service layer is **highly feasible and structurally excellent**.

Because Gleam compiles to clean, zero-dependency, idiomatic JavaScript (ES modules), Bun can execute it natively at maximum speed. This hybrid architecture gives you the best of both worlds: Hono's ultra-fast routing and massive npm ecosystem, paired with Gleam's bulletproof, type-safe functional domain logic.

How the Architecture Works

When compiling to JavaScript, Gleam preserves its type guarantees and outputs pure modules.

[HTTP Request]



1. **The Web Layer (Bun/Hono):** Acts as the public face of your app. It handles HTTP parsing, CORS, cookies, validation (via Zod), and middleware (Auth).
2. **The Interop Boundary:** You import the compiled JavaScript files generated by Gleam directly into your TypeScript/JavaScript Hono routes.
3. **The Service Layer (Gleam):** Houses your core business logic, data transformations, domain validation, and pure algorithms.

Strengths of this Hybrid Approach

1. Zero-Overhead Performance

When Gleam targets JavaScript, there is no virtual machine emulation or heavy runtime layer. A Gleam function becomes a standard JavaScript function. Bun executes it using its highly optimized **JavaScriptCore engine**, meaning the interop cost between Hono and Gleam is **exactly zero**.

2. Robust Domain Modeling via Gleam

You can isolate your most complex business rules in Gleam. Gleam's compiler strictly enforces that you handle every edge case (no null pointer exceptions, no unhandled exceptions, and mandatory pattern matching on custom types). Your core service layer becomes completely deterministic and practically bug-free.

3. Seamless Build Integration

Bun natively runs JavaScript and TypeScript out of the box. To make this setup work, you simply run Gleam's compiler in watch mode alongside Bun:

- `gleam build --target=javascript` compiles your code into a local `build/dev/javascript` directory.
- Your Hono code imports directly from that build folder.

⚠ Challenges & Friction Points

While feasible, combining these two ecosystems creates a few developer-experience hurdles you must manage:

1. The TypeScript / Gleam Type Gap

Gleam does not automatically generate TypeScript `.d.ts` declaration files out of the box. When you import a compiled Gleam module into your TypeScript Hono app, TypeScript will view it as an untyped any module unless you manually declare the types or use `@ts-ignore`.

- *The Fix:* To keep your Hono code completely type-safe, you must write a thin TypeScript wrapper interface around the imported Gleam code, or use Gleam's external functions carefully.

2. Data Structure Conversion

Gleam uses its own highly optimized functional data structures when compiled to JS (e.g., Gleam lists are not standard JS arrays; they are linked lists).

- If a Gleam service returns a `List(String)`, Hono cannot immediately send that as a JSON array response.

- *The Fix:* You must use Gleam's built-in `gleam/javascript` library to explicitly convert Gleam data structures into standard JavaScript Arrays and Objects before returning them to the Hono layer.

3. Giving up the Erlang Actor Model

By compiling Gleam to JavaScript, you lose the Erlang BEAM runtime. You will **not** have access to Erlang processes, supervisors, or OTP actors. Your Gleam code will run on Bun's single-threaded async event loop. For a service layer, this is usually perfectly acceptable, but it is a critical distinction to remember.

Example Setup

1. The Gleam Service (`src/service.gleam`):

```
gleam
```

```
import gleam/javascript/array.{type Array}
```

```
// Pure business logic with strict type safety
```

```
pub fn calculate_discount(price: Float, user_tier: String) -> Float {
```

```
  case user_tier {
```

```
    "VIP" -> price *. 0.80
```

```
    "Premium" -> price *. 0.90
```

```
    _ -> price
```

```
  }
```

```
}
```

Use code with caution.

2. The Hono Route (`server.ts`):

```
typescript
```

```
import { Hono } from 'hono'
```

```
// Importing the compiled ES module output by Gleam
```

```
// @ts-ignore (or use a local .d.ts file)

import { calculate_discount } from './build/dev/javascript/my_project/service.mjs'

const app = new Hono()

app.get('/checkout', (c) => {

  const price = 100.00

  const tier = "VIP"

  // Call the bulletproof Gleam logic inside your fast Bun server

  const finalPrice = calculate_discount(price, tier)

  return c.json({ finalPrice })

})

export default app
```

Use code with caution.

To see if this architecture fits your specific workflow, tell me:

- Will your **Gleam service layer** need to talk directly to the **database**, or should Hono handle data-fetching and pass raw objects into Gleam?
- Are you comfortable managing the **TypeScript definition mapping** for the compiled Gleam files, or would you prefer a strategy that minimizes TypeScript overhead?

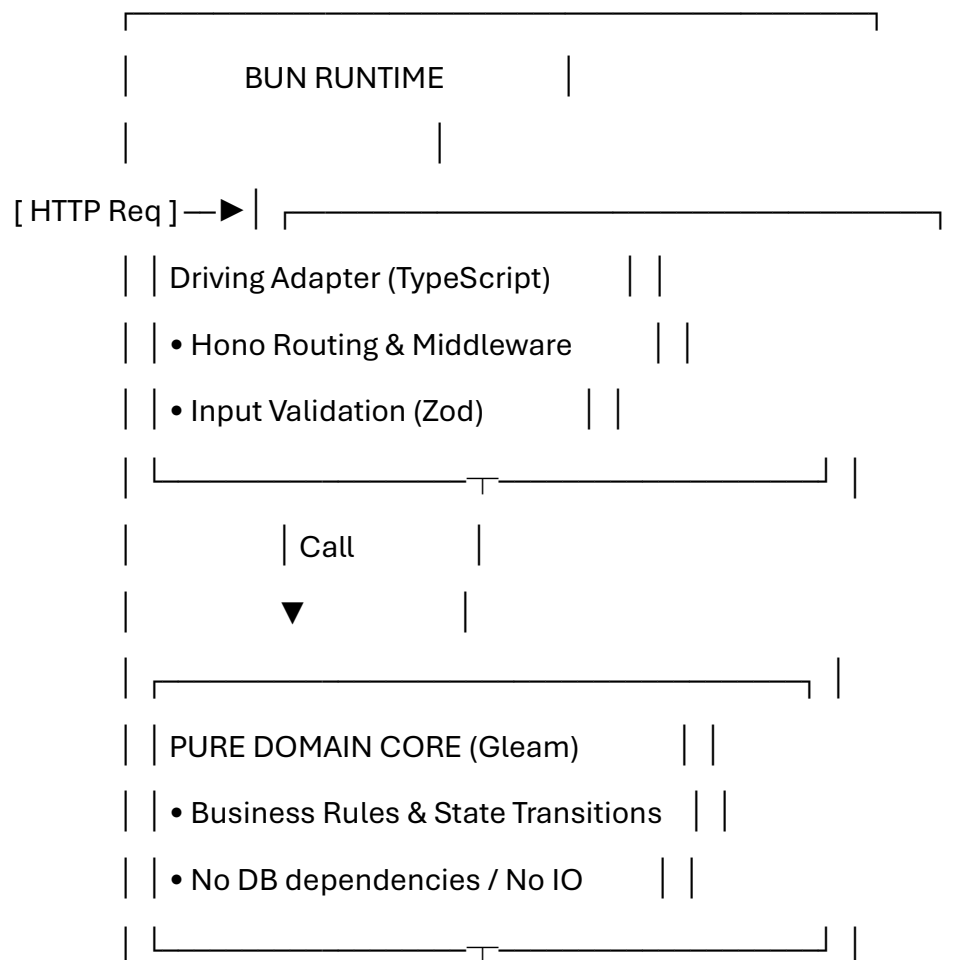
Evaluate: domain logic in gleam, web ingress and postgresql persistence in typescript - hexagonal architecture style almost enforced by the language crossovers

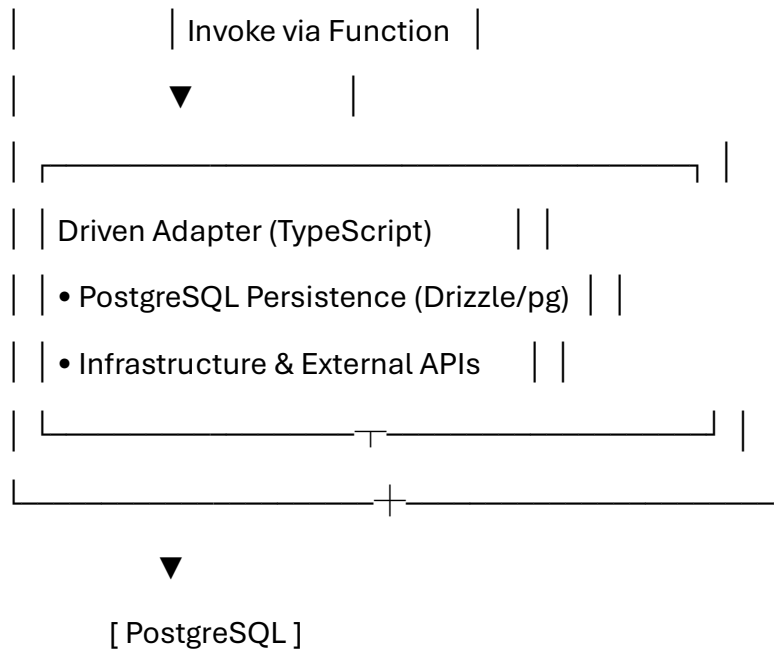
This architectural split is a **brilliant, highly pragmatic application of Hexagonal Architecture (Ports and Adapters)**. By forcing the language boundary to serve as the architectural boundary, you are creating a system where it is **physically difficult to write low-quality, tightly coupled code**.

Instead of relying on developer discipline to keep business rules clean, the **compiler and runtime ecosystem enforce it for you**.

The Language-Enforced Hexagonal Blueprint

In classic Hexagonal Architecture, the core domain should have zero knowledge of databases, HTTP routers, or third-party protocols. Your setup makes this absolute:





● The Good: Why This Setup Excels

1. Un-compromised Domain Purity

In single-language TypeScript projects, developers routinely leak ORM models (like Prisma or Drizzle objects) directly into the business logic. Because **Gleam cannot easily or natively read complex TypeScript ORM classes**, you are forced to map incoming database rows into pure Gleam custom types (Records). Your domain logic remains perfectly decoupled, deterministic, and trivial to unit-test.

2. Best-of-Breed Tooling Selection

- **The I/O Layer (TypeScript):** Database connections, connection pooling, migrations, and HTTP parsing are messy, infrastructure-heavy tasks. TypeScript excels here because of tools like **Drizzle ORM, Zod, and Hono**, which have massive communities and robust edge cases ironed out.
- **The Logic Layer (Gleam):** Business rules are full of edge cases and conditional branches. Gleam's **exhaustive pattern matching** and **sound type system** ensure that your core calculations are 100% bug-free, eliminating undefined is not a function bugs at the core.

3. Trivial, Mock-Free Unit Testing

Because your Gleam layer cannot perform side effects (no direct Postgres queries), testing your core business rules requires no database mocking libraries, no containers, and no complex test environments. You simply pass raw data structures to Gleam functions and assert on the outputs.

● The Bad: Architectural Friction Points to Watch Out For

While structurally beautiful, your day-to-day coding will face specific frictions introduced by the language boundary:

1. The Interop Serialization Tax

A standard request-response loop will require significant data mapping:

1. **Hono:** Receives raw JSON $\rightarrow$ Validates via Zod $\rightarrow$ Converts to clean JS object.
2. **Boundary:** You must map the JS object into a format the compiled Gleam code understands (converting JS arrays to Gleam lists, strings to Gleam atoms/strings, etc.).
3. **Gleam Core:** Executes logic $\rightarrow$ Returns a Gleam custom type or Result.
4. **Boundary:** Convert Gleam structures back to standard JS arrays/objects.
5. **Persistence (TS):** Save the mapped JS object to Postgres via Drizzle.

This results in a lot of boilerplate code whose sole purpose is translating data between TypeScript and Gleam.

2. Managing the Driven Adapter (Database Fetching)

If your Gleam domain logic requires database data halfway through a calculation (e.g., checking if a user has enough balance before processing an order), you have two choices:

- **The Anemic Approach:** Fetch *all* potentially relevant database rows in TypeScript first, then pass them en masse into the Gleam function.
 - **The Dependency Injection (Ports) Approach:** Define a function type in Gleam (the Port) that expects a callback. In TypeScript, pass a function that queries Postgres (the Adapter) into Gleam. While architecturally correct, executing JavaScript async/await promises inside compiled Gleam functions can get syntactically greasy.
-

Execution Strategy to Minimize Friction

If you choose to implement this structure, use these rules of thumb to maximize productivity:

1. **Keep Gleam Completely Synchronous:** Avoid handling async operations (Promises) inside Gleam if possible. Let TypeScript do all the async/await heavy lifting to fetch data from Postgres, and pass raw data into synchronous Gleam functions.
2. **Automate the Type Bridge:** Write a small script or use toolings to generate simple TypeScript .d.ts declaration files for your compiled Gleam build/ files, or isolate all your Gleam call sites to a single, tightly controlled domain-bridge.ts file so @ts-ignore doesn't leak everywhere.
3. **Use Primitive Boundaries:** When passing data across the boundary, try to flatten your data into primitive JS arrays and objects using Gleam's gleam/javascript module before returning them to TypeScript.

To weigh the development trade-offs, tell me:

- Does your domain logic rely heavily on **frequent, complex database lookups mid-calculation**, or is it mostly linear data transformation?
- Are you aiming for an **event-driven setup** (e.g., background queues) or a strict **Request/Response API** layout?